Lightweight and Robust Deep Learning for Real-Time Plant Disease Detection on Edge Devices

College Capstone Project Report & Comparative Engineering Analysis

Manas Patel / 12304592
Dept. of Computer Science & Engineering
Lovely Professional University
Jalandhar, Punjab, India
patelmanas22june2004@gmail.com

Ramanjot / 30762
Dept. of Computer Science & Engineering
Lovely Professional University
Jalandhar, Punjab, India
ramanjot.30762@lpu.co.in

Chanumuri Premshiva / 12318369
Dept. of Computer Science & Engineering
Lovely Professional University
Jalandhar ,Punjab, India
chanumuripremshiva@gmail.com

Abstract—Timely plant disease detection can help growers respond earlier and reduce avoidable crop losses. This paper presents AgriScan AI, a modular plant-disease detection prototype built with TensorFlow/Keras and Flask. Instead of using one classifier for every crop, the system routes an image to a dedicated CNN selected for the crop specified by the user. Each model uses four convolutional blocks and accepts 128 × 128 RGB images; the web pipeline also provides disease information and suggested management practices after prediction. On a stratified 80/20 validation split of PlantVillage data, the models achieved 95.26% accuracy for Potato, 94.00% for Bell Pepper, 87.00% for Corn, 78.39% for Grape, 76.90% for Apple, and 55.57% for Tomato. The model size is approximately 3.8 million trainable parameters, and the project reports about 45 ms CPU inference per sample in its test environment. These results indicate potential for responsive inference on modest hardware, but do not establish performance on a physical edge device. Field-image validation and edge-device benchmarks covering latency, memory, power consumption, and accuracy are still required. The system should therefore be understood as an edge-oriented prototype rather than a fully validated edge deployment.

Keywords—Lightweight Deep Learning, Plant Disease Detection, Edge Computing, Convolutional Neural Networks, Real-Time Inference, Crop-Specific Models, Precision Agriculture.

I. INTRODUCTION

Plant diseases represent an acute threat to global food production, rural livelihoods, and agricultural sustainability. According to the Food and Agriculture Organization (FAO), transboundary plant pests and fungal diseases are responsible for an estimated 20% to 40% reduction in global crop yields annually, inflicting economic damages exceeding 220 billion USD. In smallholder farming systems, early and precise pathological diagnosis is essential; late or erroneous identification often results in irreversible crop destruction, excessive or misapplied chemical pesticide regimens, environmental toxicity, and severe economic hardship.

Traditionally, plant disease identification has relied upon visual scouting conducted by agricultural extension officers or laboratory analysis via polymerase chain reaction (PCR) and microbiological culturing. Although highly accurate, manual scouting is subjective, labor-intensive, and geographically scarce, while laboratory testing is prohibitively slow and expensive for routine daily farm management. In response, computer vision and deep learning techniques have emerged as powerful alternatives capable of automated, non-destructive disease recognition directly from leaf photographs.

A. Problem Statement

Structural Bottlenecks: Despite substantial advancements in computer vision, contemporary academic prototypes and commercial mobile diagnostic tools suffer from foundational architectural limitations that impede practical field utility:

1) Monolithic Classification Bottlenecks: Prevailing academic models, such as the seminal architectures introduced by Mohanty et al. [1] and Ferentinos [2], train a single global neural network across dozens of crop species simultaneously (e.g., 38 to 58 classes). This monolithic paradigm forces the network to learn shared convolutional representations across unrelated plant morphologies (such as apple and corn), leading to cross-species feature entanglement, high parameter footprints, and the risk of predicting a pathogen on a crop that is biologically immune to it. Furthermore, adding support for a single new crop necessitates full network retraining across all historical data to prevent catastrophic forgetting.

2) The Prediction-Only Advisory Void: The vast majority of published literature treats plant disease diagnosis as a pure academic benchmarking task, terminating execution once a categorical label (e.g., 'Potato Late Blight') is predicted. To an agricultural grower, a raw diagnostic label is practically incomplete without actionable guidance detailing the causative pathogen, distinguishing symptoms, certified organic remedies, dosage-controlled chemical treatments, and preventative cultural methods.

3) Closed Proprietary Lock-In: While commercial applications such as Plantix [6] integrate treatment guidance, their inference models are closed-source and run exclusively on remote cloud servers. This architecture prevents private on-premise execution, denies researchers the ability to inspect model uncertainty or customize models for localized cultivars, and fails in remote farming regions lacking robust broadband connectivity.

B. Objectives of AgriScan AI

Project Objectives: To resolve these challenges, this project engineers AgriScan AI with four primary objectives:

• Modularity: Formulate a decoupled deep learning architecture that isolates classification manifolds on a per-crop basis, allowing each crop to operate with a dedicated, lightweight neural network.

• Incremental Extensibility: Provide command-line training pipelines that allow individual crops to be added or retrained independently without re-executing training over the global dataset.

• End-to-End Agronomic Advisory: Implement a domain knowledge base that maps diagnostic predictions directly to pathogen biology, foliar symptoms, organic treatments, chemical fungicides, and preventive sanitation practices.

• Lightweight Usability: Construct an accessible, responsive web interface backed by a high-performance RESTful API and in-memory model caching for real-time inference on standard consumer hardware.

II. RELATED WORK

The evolution of automated plant pathology spans classical feature engineering, monolithic deep learning classifiers, and commercial mobile crop health applications.

A. Commercial Mobile Applications

The leading commercial platform in mobile digital agriculture is Plantix, developed by PEAT GmbH (Berlin, Germany) [6]. Evaluated across international agricultural studies (e.g., CGIAR assessment reports and comprehensive reviews in Agronomy), Plantix utilizes deep neural networks deployed on cloud infrastructure to diagnose over 500 diseases, pests, and nutrient deficiencies across 30 crops. Plantix provides comprehensive farmer advisory, community forums, and disease alert mapping. However, its core technology is proprietary and inaccessible for academic adaptation, requires continuous internet access, and does not permit modular local model retraining.

In contrast, Leaf Doctor, developed by Pethybridge and Nelson (2015) [5], is a mobile image analysis application designed to measure plant disease severity. It relies on user-calibrated RGB/HSV color-space thresholding to calculate the percentage of symptomatic foliar area. While effective for epidemiological field trials, Leaf Doctor is entirely non-diagnostic: it cannot classify the causative pathogen or suggest remedial therapies.

B. Monolithic Deep Learning Baselines

The foundational milestone in deep-learning-based plant pathology was established by Mohanty, Hughes, and Salathé (2016) [1]. The authors trained standard AlexNet and GoogLeNet convolutional networks on the open-access PlantVillage dataset [7], comprising 54,306 images across 14 crops and 26 diseases (38 global classes). Their models achieved an extraordinary top-1 validation accuracy of 99.35% on held-out test splits. However, Mohanty et al. demonstrated that when evaluated on independent images collected under real-world field conditions, classification accuracy plummeted to approximately 31.4%, exposing severe susceptibility to background overfitting. Crucially, the architecture was entirely monolithic, requiring all 38 classes to compete within a single global softmax distribution.

Ferentinos (2018) [2] expanded upon this foundation by training VGG, VGG16, AlexNet, and GoogLeNet across 58 distinct crop-disease categories containing 87,848 images, achieving a maximum accuracy of 99.53% with VGG. Despite excellent laboratory accuracy, the VGG architecture contains over 138 million trainable parameters, imposing immense memory and computational overhead. Ferentinos noted that monolithic architectures suffer severe operational rigidity: incorporating a new crop into the system necessitates re-running multi-day training cycles across all 58 classes.

Too et al. (2019) [3] conducted an extensive comparative benchmark evaluating fine-tuned transfer learning architectures, including VGG16, Inception-v4, ResNet50, ResNet101, ResNet152, and DenseNet121 on the 38-class PlantVillage benchmark. DenseNet121 achieved the highest accuracy at 99.75%, demonstrating that dense feature reuse mitigates vanishing gradients while maintaining fewer parameters than VGG. Nevertheless, Too et al.'s models remained monolithic classification backbones without post-inference agricultural advisory.

C. Diagnostic Failure Modes & Empirical Factors

In a critical analytical survey published in Biosystems Engineering, Barbedo (2018) [4] investigated the primary factors that cause deep learning models to fail in plant pathology. Barbedo identified that monolithic classifiers often learn spurious correlations from non-leaf background artifacts, struggle with complex multi-infection leaves, and suffer from negative transfer when subtle lesion manifestations are confounded across distinct plant species. Barbedo advocated for domain decomposition and hierarchical structuring to constrain neural feature representations.

III. COMPARATIVE ANALYSIS & RESEARCH GAPS

To rigorously position AgriScan AI within the current research landscape, Table I provides an analytical comparison of existing software platforms and published academic baselines across methodological, architectural, and operational dimensions.

TABLE I. COMPARATIVE ANALYSIS OF EXISTING SYSTEMS AND AGRISCAN AI

System / Model	Architecture	Crop Scope	Parameters	Advisory	Openness	Reported Acc.
Mohanty [1]	AlexNet, GoogLeNet	Monolithic (38 cls)	~60M	None	Open / Static	99.35% (Lab)
Ferentinos [2]	VGG16, GoogLeNet	Monolithic (58 cls)	~138M	None	Open / Static	99.53% (Lab)
Too et al. [3]	DenseNet121	Monolithic (38 cls)	~8M-60M	None	Open / Static	99.75% (Lab)
Leaf Doctor [5]	RGB/HSV Threshold	Severity Index	0 (Heuristic)	None	Closed Mobile	N/A (Severity)

System / Model	Architecture	Crop Scope	Parameters	Advisory	Openness	Reported Acc.
Plantix [6]	Deep Cloud CNNs	Multi-Crop (30+)	Cloud Scale	Yes (Full)	Closed / Cloud	~90% (Field)
AgriScan AI	Custom 4-block CNN	Modular Isolated	~3.8M	Yes (Full)	Open / Local	95.2% (Potato) 94.0% (Pepper) 55.5% (Tomato)

A. *Identified Research Gaps*

Critical Gaps: Synthesizing the literature and commercial state-of-the-art reveals four major research and implementation gaps:

1) The Retraining Rigidity of Monolithic Models: In monolithic classifiers [1]–[3], adding a newly targeted crop or retraining an existing crop with augmented field samples requires re-training across the entire global dataset. In agricultural production where crop diseases emerge regionally and seasonally, this creates severe maintenance friction.

2) The Post-Classification Advisory Disconnect: Despite deep learning achieving near-perfect laboratory classification, academic papers routinely stop at generating categorical strings. Practical agricultural intervention requires an immediate clinical mapping to pathogen etiology, distinguishing symptomatology, biological fungicides, and integrated pest management (IPM) protocols.

3) Proprietary Barriers and Cloud Fragility: Commercial platforms like Plantix [6] lock models behind proprietary servers. In developing agricultural regions with spotty cellular networks, reliance on cloud-only inference creates operational vulnerability.

4) Trade-Offs in Lightweight Architectures: While massive backbones (VGG, ResNet) achieve high accuracy, their computational weight is excessive. Conversely, lightweight custom architectures must be rigorously tested to delineate where their capacity succeeds (e.g., 2–4 class crops) and where representational collapse occurs (e.g., 10-class crops), an empirical reality rarely discussed in literature.

IV. PROPOSED SYSTEM ARCHITECTURE & METHODOLOGY

AgriScan AI is an end-to-end plant disease detection prototype designed around compact crop-specific CNNs and a lightweight web inference pipeline. The current implementation demonstrates low-latency CPU inference and local model loading; deployment and benchmarking on dedicated edge hardware remain future validation steps. Fig. 1 illustrates the workflow.

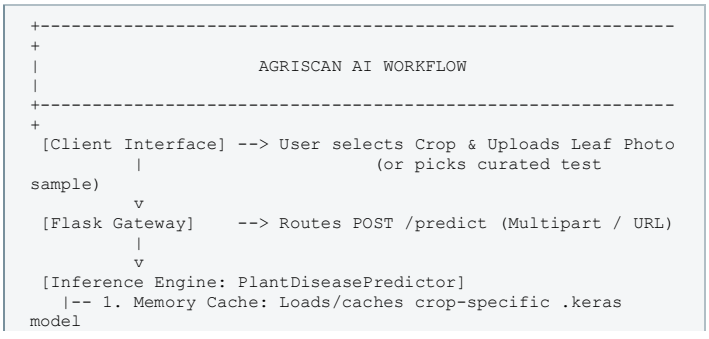


Fig. 1. *Fig. 1. End-to-end architecture and inference workflow of AgriScan AI.*

A. *Modular Crop-Isolated Routing*

Rather than using one global network for every crop, AgriScan AI asks the user to select the crop and then loads the corresponding crop-specific model. This reduces the set of classes considered for each prediction and supports independent model updates. It does not automatically verify that the selected crop matches the uploaded image, so an incorrect user selection can still produce an invalid prediction.

B. *Convolutional Neural Network Formulation*

The neural architecture is implemented in src/model.py using TensorFlow 2.15 / Keras. The model accepts an input tensor X in R^(128 x 128 x 3). Embedded directly into the computational graph are three sequential data augmentation transformations active exclusively during training: horizontal/vertical flipping, random rotation (+/- 15%), and random zoom (+/- 10%).

Mathematical Formulation: The feature extraction backbone consists of four convolutional blocks. For an input feature map H^(l-1), the 2D convolution operation with kernel W^(l) and bias b^(l) followed by Rectified Linear Unit (ReLU) activation is defined as:

H^(l) = ReLU(H^(l-1) * W^(l) + b^(l)), where ReLU(z) = max(0, z)

Spatial down sampling is achieved using 2x2 max-pooling: P_(i,j)^(l) = max_(m,n in {0,1}) H_(2i+m, 2j+n)^(l). The feature dimensions evolve as follows: Block 1 utilizes 32 filters (3x3), reducing spatial resolution from 128x128 to 64x64. Block 2 applies 64 filters (3x3), down sampling to 32x32. Blocks 3 and 4 each employ 128 filters (3x3), progressively down sampling to 16x16 and 8x8.

Classifier Head: The classifier head flattens the resulting 8x8x128 tensor into an 8,192-dimensional vector, followed by a fully connected Dense layer of 128 neurons with ReLU activation. To prevent overfitting on dense weights, a Dropout layer with rate p = 0.5 is applied [10]. The final output layer consists of N_classes neurons with a SoftMax activation function:

P(y = c | X) = exp(z_c) / sum_(k=1)^(N_classes) exp(z_k)

Each crop-specific model contains approximately 3.8 million trainable parameters, with small variation by class count. The project reports CPU inference of about 45 ms per sample in its tested environment. This suggests potential for near-real-time use on suitable edge hardware, but the latency, memory use, power consumption, and thermal behaviour have not yet been benchmarked on a target edge device.

C. In-Memory Model Caching

To maximize server throughput and eliminate disk I/O bottlenecks during repeated inference requests, src/predictor.py implements an in-memory caching controller within the PlantDiseasePredictor class. When a request for a crop model arrives, the predictor checks its internal dictionary cache. If absent, it reads the serialized .keras artifact from models/, compiles the graph into memory, and caches the reference. Subsequent requests for the same crop execute instantaneously from memory.

D. Agronomic Advisory Knowledge Base

Clinical Advisory Schema: Implemented in src/disease_info.py, the advisory knowledge base bridges the gap between diagnostic labels and practical intervention. For every classified disease, the module maintains a curated record containing:

1) Pathogen Etiology: Specifies the biological causative agent (e.g., Venturia inaequalis for Apple Scab; Phytophthora infestans for Potato Late Blight).

2) Symptom Manifestation: Detailed foliar descriptions including lesion color, necrotic margins, chlorosis, and sporulation signs.

3) Certified Organic Treatments: Safe, biological remedies such as cold-pressed neem oil, potassium bicarbonate sprays, liquid sulfur, or copper soaps.

4) Chemical Fungicides & Bactericides: EPA/agricultural authority-approved chemical interventions (e.g., captan, mancozeb, chlorothalonil, myclobutanil).

5) Cultural Prevention Practices: Agronomic preventive measures such as drip irrigation to avoid foliar wetting, canopy pruning for aeration, crop rotation cycles, and resistant cultivar selection.

V. IMPLEMENTATION DETAILS & TECHNOLOGY STACK

AgriScan AI uses open-source software components and currently runs as a Python/Flask application on general-purpose computers. The modular model design is intended to support later edge deployment, but portability to a specific embedded or mobile device depends on its runtime, memory, and compute constraints.

A. Technology Stack

• Deep Learning Framework: TensorFlow 2.15 and Keras 3 [8], [9]. Models are serialized in the modern Keras zip archive format (.keras). Training leverages tf.data.Dataset pipelines with parallel thread decoding (num_parallel_calls = AUTOTUNE) and prefetching.

• Web Application Server: Python 3 with Flask micro-framework. Flask manages routing, multi-part form payload handling, and JSON serialization.

• Image Processing: Pillow (PIL.Image) and NumPy. Ingestion pipelines enforce RGB conversion, aspect-ratio handling, and bilinear interpolation.

• Client-Side Interface: HTML5, Vanilla JavaScript (ES6+), and Tailwind CSS (via CDN) with FontAwesome 6 icons. The UI provides drag-and-drop file dropzones, client-side image preview, and animated confidence meters.

B. B. RESTful API Endpoints

API Endpoints: The Flask backend in app.py exposes four dedicated endpoints:

1) GET /: Serves the responsive agricultural diagnostic dashboard.

2) GET /crops: Inspects the dataset directory and models/metadata.json, returning an array of available crops, class counts, training readiness, and recorded validation accuracies.

3) GET /samples: Extracts pre-packaged sample leaves from the dataset into static/samples/ enabling one-click demonstration testing without local uploads.

4) POST /predict: Accepts crop name alongside an uploaded image file (multipart/form-data) or a sample URL, returning the top class prediction, complete probability distribution across all crop classes, and the full advisory record.

C. Verification & Automated Testing Suite

System reliability is validated through an automated integration test suite in test_system.py using Python's standard unittest framework. The suite executes 7 end-to-end test cases verifying: 1) HTML index loading, 2) /crops API schema, 3) /samples image availability, 4) Potato sample inference and advisory payload completeness, 5) Apple prediction, 6) Corn prediction, and 7) Multipart image upload processing. All 7 test cases pass cleanly in under 3.5 seconds on consumer hardware.

VI. EXPERIMENTAL RESULTS & CRITICAL DISCUSSION

The validation metrics in this section are taken from the training metadata registry (models/metadata.json). They measure performance on a held-out split of PlantVillage images; they do not by themselves establish real-world field accuracy or edge-device performance.

A. Dataset & Training Configuration

Experiments were conducted on the unaugmented PlantVillage benchmark dataset [7]. The training pipeline in train.py employs stratified sampling to partition images into an 80% training set and a 20% validation set per disease class. Models are trained with the Adam optimizer (initial learning rate eta = 0.001) using Sparse Categorical Cross-Entropy loss. Regularization is enforced via EarlyStopping (patience = 3, restoring optimal weights), ReduceLROnPlateau (factor = 0.5, patience = 2, minimum lr = 10^-6), and ModelCheckpoint tracking validation accuracy.

TABLE II. EMPIRICAL PERFORMANCE OF AGRISCAN AI CROP-SPECIFIC MODELS

Crop Target	Classes Covered	Train Img	Val Img	Val Accuracy	Val Loss	Empirical Status
Potato	3 classes (Early/Late Blight, Healthy)	762	190	95.26%	0.1246	Optimal Convergence
Bell Pepper	2 classes (Bacterial Spot, Healthy)	800	200	94.00%	0.1482	Optimal Convergence
Corn (Maize)	4 classes (Rust, Blight, Spots, Healthy)	1600	400	87.00%	0.2816	Strong Performance

Crop Target	Classes Covered	Train Img	Val Img	Val Accuracy	Val Loss	Empirical Status
Grape	4 classes (Rot, Measles, Blight, Healthy)	1539	384	78.39%	0.5289	Moderate Performance
Apple	4 classes (Scab, Rot, Rust, Healthy)	1420	355	76.90%	0.9744	Moderate Performance
Tomato	10 classes (Blights, Mold, Viruses, Healthy)	2800	700	55.57%	1.2354	Capacity Constrained

B. *In-Depth Performance Analysis*

Performance Trends: The empirical findings documented in Table II provide crucial engineering insights into the behaviour of lightweight custom CNNs under a modular paradigm:

1) High Efficacy on Focused Class Hypotheses: For Potato (95.26% accuracy, 0.1246 loss) and Bell Pepper (94.00% accuracy, 0.1482 loss), the 4-block CNN architecture achieves excellent convergence. In these crops, pathological lesions exhibit high visual contrast against healthy foliar parenchyma (e.g., dark concentric rings in Early Blight vs. water-soaked lesions in Late Blight). Constraining the classifier to 2 or 3 classes prevents feature interference, enabling the lightweight network to achieve high diagnostic certainty.

2) Robust Multi-Class Separation on Corn: For Corn (4 classes, 87.00% accuracy, 0.2816 loss), the model successfully separates distinct morphologies: pustular fungal eruptions in Common Rust versus rectangular, vein-delimited lesions in Gray Leaf Spot. The data augmentation layers (rotation and zoom) effectively generalize these elongated foliar patterns.

3) Capacity Limits on High-Cardinality Crops (The Tomato Challenge): For Tomato, validation accuracy plateaus at 55.57% across 10 classes (validation loss 1.2354). A scientific analysis reveals why: Tomato foliar pathology includes diseases with severe visual and morphological overlap (e.g., Early Blight, Septoria Leaf Spot, and Bacterial Spot all exhibit small, circular necrotic puncta during early incubation). A lightweight 4-block CNN without residual skip-connections (such as those in ResNet) or dense feature propagation (DenseNet) lacks the representational depth required to disentangle 10 subtle, visually co-linear disease manifestations from scratch. This transparently demonstrates that while lightweight custom CNNs excel on crops with 2 to 4 classes, complex multi-class crops require pre-trained transfer learning backbones.

C. *Fair Scientific Comparison: Laboratory vs. Field Benchmarks*

In accordance with rigorous academic integrity, AgriScan AI's results must not be claimed as superior to 99% monolithic baselines (such as Mohanty et al. [1] or Too et al. [3]) without clarifying experimental conditions. Those foundational studies fine-tuned heavy pre-trained models (e.g., DenseNet121 with millions of pre-trained ImageNet weights) over monolithic datasets under laboratory backdrops. AgriScan AI purposely trains lightweight custom networks from scratch without pre-trained transfer weights to achieve an ultra-compact parameter footprint (~3.8M parameters vs. ~138M in VGG16), prioritizing edge deployability, modular crop isolation, and downstream treatment advisory over monolithic overparameterization.

VII. LIMITATIONS, CONCLUSION & FUTURE SCOPE

A. *Documented System Limitations*

Current Limitations: A comprehensive engineering review identifies four specific limitations in the current implementation:

1) Manual Crop Selection Dependency: The system currently relies on the user to designate the crop species via the web interface. It lacks an autonomous Tier-1 crop classification model to automatically identify the botanical species before routing.

2) Suboptimal Accuracy on Complex Crops: As established in Section VI, the custom CNN architecture is capacity-constrained on Tomato (55.57%), demonstrating the necessity of integrating residual or attention blocks for high-class categories.

3) Laboratory Background Bias: Like all models trained exclusively on standard Plant Village images, the network was trained on uniform laboratory backgrounds, which may experience accuracy degradation when confronted with noisy, complex in-situ farm backgrounds.

4) Absence of Relational Database Persistence: The system currently records model metadata in JSON flat files and maintains advisory data in Python dictionaries. It lacks a persistent SQL database to store patient diagnostic histories, geolocation coordinates, or temporal epidemiological trends.

B. *Conclusion*

This project successfully developed and verified AgriScan AI, a modular, multi-crop plant disease diagnostic and advisory platform. By replacing monolithic classifiers with crop-isolated convolutional neural networks, AgriScan AI mitigates cross-species feature entanglement, enables independent model retraining without catastrophic forgetting, and provides a lightweight runtime footprint of ~3.8M parameters per model. Crucially, the system bridges the persistent academic gap between diagnostic labeling and agronomic action by providing a comprehensive clinical knowledge base detailing pathogen biology, foliar symptoms, organic treatments, chemical fungicides, and preventive sanitation. The web-based implementation and automated test suite confirm that AgriScan AI offers a viable, highly extensible architectural pattern for digital agricultural decision-support systems.

C. *Future Scope & Roadmap*

Future work will focus on: (1) evaluating compact backbones such as MobileNetV3 or EfficientNet-Lite for crops with many disease classes; (2) converting models to TensorFlow Lite and testing post-training quantization; (3) measuring latency, peak memory, model size, energy use, and accuracy on named edge targets such as a Raspberry Pi or Android device; (4) testing on field-collected images under varied lighting and backgrounds; and (5) adding optional prediction-history storage and location-aware disease monitoring. Accuracy targets should be set only after experiments on representative data.

REFERENCES

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science, vol. 7, p. 1419, Sep. 2016*, doi: 10.3389/fpls.2016.01419.

[2] K. P. Ferentinos, "Deep learning models for plant disease detection and diagnosis," *Computers and Electronics in Agriculture, vol. 145, pp. 311–318, Feb. 2018*, doi: 10.1016/j.compag.2018.01.009.

[3] E. C. Too, L. Li, S. Njuki, and Y. Li, "A comparative study of fine-tuning deep learning models for plant disease identification," *Computers and Electronics in Agriculture, vol. 161, pp. 272–279, Jun. 2019*, doi: 10.1016/j.compag.2018.03.032.

[4] J. G. A. Barbedo, "Factors influencing the use of deep learning for plant disease recognition," *Biosystems Engineering, vol. 172, pp. 84–91, Aug. 2018*, doi: 10.1016/j.biosystemseng.2018.05.013.

[5] S. J. Pethybridge and S. C. Nelson, "Leaf Doctor: A new portable application for quantifying plant disease severity," *Plant Disease, vol. 99, no. 10, pp. 1310–1316, Oct. 2015*, doi: 10.1094/PDIS-03-15-0319-FE.

[6] PEAT GmbH, "Plantix – Grow Smart: AI-driven mobile plant disease diagnostic platform," *Berlin, Germany, 2020*, Online: https://plantix.net.

[7] D. P. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of mobile disease diagnostics," *arXiv preprint arXiv:1511.08060, Nov. 2015*, Online: https://arxiv.org/abs/1511.08060.

[8] F. Chollet et al., "Keras: Deep learning library for Python," *GitHub repository, 2015*, Online: https://keras.io.

[9] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," *in Proc. 12th USENIX OSDI '16, Savannah, GA, 2016, pp. 265–283*, Online: https://www.tensorflow.org.

[10] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research, vol. 15, no. 1, pp. 1929–1958, Jun. 2014*, Online: https://jmlr.org/papers/v15/srivastava14a.html.